

Introduction to AI - Uncertainty

Author: Refat Othman

Learning Objectives

By the end of this lecture, we will be able to:

- Understand why uncertainty is fundamental in AI
- Define probability formally using possible worlds
- Work with unconditional and conditional probability
- Understand and test independence
- Apply Bayes’ Rule with intuition and mathematics
- Use joint probability, marginalization, and conditioning
- Model problems using Bayesian Networks
- Perform exact and approximate inference
- Understand uncertainty over time (Markov Models & HMMs)

1. Motivation: Why Uncertainty in AI?

In the real world, AI systems rarely have access to complete and perfect information. Sensors are noisy, data may be missing, and the environment may change.

Real-World Examples

- Weather forecasting: Will it rain tomorrow?
- Medical diagnosis: Does the patient have a disease given a test result?
- Spam detection: Is this email spam?
- Autonomous driving: Is the detected object a pedestrian or a shadow?

Deterministic vs Probabilistic Systems

Deterministic	Probabilistic
True / False	Degree of belief
Logic rules	Probability
Perfect information	Incomplete / noisy information
Chess	Weather forecast

Key idea: AI must reason under uncertainty using probability, not just logic.

2. Probability and Possible Worlds

2.1 Possible Worlds

A **possible world** is a complete description of how the world could be.

Example: Rolling Two Dice

- Each die has values {1, 2, 3, 4, 5, 6}
- Total possible worlds = $6 \times 6 = 36$
- Example world: (2, 5)

Each possible world represents one outcome of the experiment.

2.2 Probability Function

A **probability function** assigns a numerical value to each possible world, representing how likely that world is.

Intuitively, probability answers the question:

How strongly do we believe that this specific world will occur?

Formal Definition

Let Ω be the set of all possible worlds. A probability function P assigns a value $P(\omega)$ to each world $\omega \in \Omega$ such that:

- **Non-negativity:** $0 \leq P(\omega) \leq 1$ for every ω
- **Normalization:** $\sum P(\omega) = 1$

These two rules ensure that probabilities are meaningful and consistent.

Why These Rules Matter (Intuition)

- A probability cannot be negative (you cannot believe something "less than impossible").
- A probability cannot be greater than 1 (you cannot be more than certain).
- The sum must be 1 because **one of the possible worlds must happen**.

If the sum were less than 1 $\rightarrow$ some outcomes are missing. If the sum were greater than 1 $\rightarrow$ probabilities are exaggerated.

Example 1: Fair Coin

Possible worlds:

$\Omega = \{\text{Heads}, \text{Tails}\}$

Probability function:

$P(\text{Heads}) = 0.5$ $P(\text{Tails}) = 0.5$

Check rules:

- Each probability is between 0 and 1 ✓
 - $0.5 + 0.5 = 1$ ✓
-

Example 2: Biased Coin

Suppose a coin is biased:

$$P(\text{Heads}) = 0.7 \quad P(\text{Tails}) = 0.3$$

This is still a valid probability function because the rules are satisfied.

Example 3: Two Dice (Revisited)

Each possible world is a pair (d_1, d_2) .

Since the dice are fair:

$$P((d_1, d_2)) = 1/36$$

Even though some events (like $\text{sum} = 7$) are more likely, **individual worlds are equally likely**.

Event Probability vs World Probability

Important distinction for students:

- **World probability:** $P((1,6)) = 1/36$
- **Event probability:** $P(\text{sum} = 7) = \text{sum of probabilities of multiple worlds}$

Formally:

$$P(\text{event}) = \sum P(\omega) \text{ for all } \omega \text{ that satisfy the event}$$

Python Example: Validating a Probability Function

```
# Example probability function for a biased coin
probability_function = {
    "Heads": 0.7,
    "Tails": 0.3
}

# Check normalization
print("Sum of probabilities:", sum(probability_function.values()))
```

Output should be:

Sum of probabilities: 1.0

Notes

1. Assigning probabilities that do not sum to 1
2. Confusing probability of a world with probability of an event
3. Thinking “more outcomes” automatically means “more probability”

Takeaway

A probability function is the **foundation of probabilistic reasoning** in AI. All later concepts (conditional probability, Bayes' rule, Bayesian networks) rely on defining valid probability functions first.

2.3 Uniform Probability Distribution

A **uniform probability distribution** is a special case of a probability distribution where **all possible worlds are equally likely**.

This means that we have **no reason to prefer one world over another**.

Formal Definition

If Ω is the set of all possible worlds and $|\Omega|$ is the number of worlds, then:

$$P(\omega) = 1 / |\Omega| \text{ for every } \omega \in \Omega$$

This guarantees:

- All probabilities are equal
- The normalization rule is automatically satisfied

Intuition (Very Important)

Uniform distribution means **complete symmetry**:

- No extra information
- No bias
- No preference

We assume uniformity when:

- A coin is fair
- Dice are fair
- Cards are well shuffled

If **any bias exists**, the distribution is **no longer uniform**.

Example 1: Fair Coin

Possible worlds:

$$\Omega = \{\text{Heads}, \text{Tails}\}$$

Uniform distribution:

$$P(\text{Heads}) = 1/2 \quad P(\text{Tails}) = 1/2$$

Why uniform?

- The coin is fair
- There is symmetry between outcomes

Example 2: Fair Die

Possible worlds:

$$\Omega = \{1, 2, 3, 4, 5, 6\}$$

Uniform distribution:

$$P(1) = P(2) = P(3) = P(4) = P(5) = P(6) = 1/6$$

Example 3: Two Dice (Important Distinction)

Possible worlds:

$$\Omega = \{(1,1), (1,2), \dots, (6,6)\}$$

Number of worlds:

$$|\Omega| = 36$$

Uniform world probability:

$$P((d_1, d_2)) = 1/36$$

Important: This does NOT mean all sums are equally likely.

Worlds vs Events (Common Exam Trap)

- Worlds are equally likely
- Events are NOT necessarily equally likely

Example:

- Event A: sum = 2 $\rightarrow$ 1 world $\rightarrow$ probability = 1/36
- Event B: sum = 7 $\rightarrow$ 6 worlds $\rightarrow$ probability = 6/36 = 1/6

Same distribution, different event probabilities.

Visual Reasoning

"Imagine all 36 outcomes written on paper. Each paper has the same weight. Events just group papers together."

Python Example: Uniform Distribution (Dice)

```
# All possible worlds for two dice
worlds = [(i, j) for i in range(1, 7) for j in range(1, 7)]

# Uniform probability distribution
uniform_prob = 1 / len(worlds)

# Assign probability to each world
P = {w: uniform_prob for w in worlds}

print("Probability of each world:", uniform_prob)
```

Python Example: Event Probability from Uniform Worlds

```
# Probability of sum = 7
prob_sum_7 = sum(P[w] for w in worlds if w[0] + w[1] == 7)

# Probability of sum = 2
prob_sum_2 = sum(P[w] for w in worlds if w[0] + w[1] == 2)

print("P(sum=7):", prob_sum_7)
print("P(sum=2):", prob_sum_2)
```

When NOT to Use Uniform Distribution

Uniform distribution is **wrong** when:

- Dice are weighted
- Coins are biased
- Cards are marked
- Prior information exists

In such cases, probabilities must be assigned differently.

Key Takeaway

Uniform probability distribution is a **modeling assumption**, not a law of nature.

It is valid only when:

- All outcomes are symmetric
- No additional information is available

Understanding this distinction is **critical for exams and real AI systems**.

2.4 Dice Example: Sum of Dice

There are 6 worlds that sum to 7: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1)

$$P(\text{sum} = 7) = 6/36 = 1/6$$

$$P(\text{sum} = 12) = 1/36$$

2.5 Python Example: Possible Worlds

```
# Generate all possible worlds for two dice
worlds = [(i, j) for i in range(1, 7) for j in range(1, 7)]

# Uniform probability distribution
P = {w: 1 / len(worlds) for w in worlds}

# Probability that the sum is 7
prob_sum_7 = sum(P[w] for w in worlds if w[0] + w[1] == 7)
print("P(sum=7):", prob_sum_7)
```

Explanation:

- We enumerate all possible worlds
 - Assign equal probability
 - Sum probabilities of worlds that satisfy the condition
-

3. Unconditional and Conditional Probability

3.1 Unconditional Probability

Unconditional probability is the degree of belief in a proposition without any additional evidence.

Examples:

- $P(\text{rain today})$
 - $P(\text{sum} = 12)$
-

3.2 Conditional Probability

Conditional probability measures belief **given** that some evidence is known.

Notation:

$P(a \mid b)$

Meaning: Probability of a given that b is true.

3.3 Conditional Probability Formula

Conditional probability formalizes how our belief in an event should **change when new information becomes available**. Instead of considering all possible worlds, we focus only on those worlds where the given evidence is true. The conditional probability formula expresses this idea mathematically by taking the probability that *both* events occur and normalizing it by the probability of the evidence alone. In other words, $P(a \mid b)$ answers the question: *among all situations where b is true, how often is a also true?* This concept is fundamental in AI because intelligent systems constantly update their beliefs as they observe new data from the environment.

$$P(a \mid b) = P(a \wedge b) / P(b)$$

Interpretation:

- Restrict attention to cases where b is true
 - Compute fraction where a is also true
-

3.4 Dice Example (Conditional)

- **$P(\text{sum} = 12) = 1/36$** This is because, when rolling two fair dice, there are **36 equally likely possible worlds** in total. The sum equals 12 **only in one specific world**, namely (6,6). Since no other combination of the two dice produces a sum of 12, the event $\{\text{sum} = 12\}$ contains exactly **one world out of 36**, which gives a probability of 1/36.
- **$P(\text{sum} = 12 \mid \text{first die} = 6) = 1/6$** Here, we are given new information: the first die is already known to be 6. This information **reduces the sample space** from 36 possible worlds to only 6 possible worlds: (6,1), (6,2), (6,3), (6,4), (6,5), (6,6). Among these six equally likely outcomes, only **one outcome**, (6,6), results in a sum of 12. Therefore, the probability becomes 1 out of 6.

Key Notes The event has not changed (sum = 12 is still only achieved by (6,6)), but **the set of possible choices has changed**. Conditional probability works by *restricting our attention to a smaller set of valid worlds* based on the given information. This is why the probability increases from 1/36 to 1/6.

Knowing the first die changes the probability space.

3.5 Python Example: Conditional Probability

```
# Worlds where first die is 6
given_first_die_6 = [w for w in worlds if w[0] == 6]
```

```
# Conditional probability
prob_sum_12_given_6 = sum(
    1 / len(given_first_die_6)
    for w in given_first_die_6
    if w[0] + w[1] == 12
)

print("P(sum=12 | first die=6):", prob_sum_12_given_6)
```

4. Random Variables and Distributions

4.1 Random Variables

A **random variable** is a variable whose value depends on the outcome of a random process.

Examples:

- Dice roll: {1,2,3,4,5,6}
- Weather: {sun, cloud, rain}
- Traffic: {none, light, heavy}

4.2 Probability Distributions

A probability distribution assigns probabilities to each value of a random variable.

Example:

$P(\text{Flight} = \text{on time}) = 0.6$ $P(\text{Flight} = \text{delayed}) = 0.3$ $P(\text{Flight} = \text{cancelled}) = 0.1$

4.3 Python Example: Distribution

```
flight_distribution = {
    "on_time": 0.6,
    "delayed": 0.3,
    "cancelled": 0.1
}

print(flight_distribution)
```

5. Independence

5.1 Core Idea (Intuition First)

Independence describes a situation where learning information about one event gives us **no new information** about another event.

In simple words:

Knowing that event **a** happened does not change how likely event **b** is.

This concept is critical in AI, because many incorrect probabilistic models fail due to **wrong independence assumptions**.

5.2 The General Multiplication Rule (ALWAYS TRUE)

For **any two events** a and b, the following rule is always correct:

$$P(a \wedge b) = P(a) \cdot P(b \mid a)$$

This formula:

- Is **always valid**
- Does **not** assume independence
- Comes directly from the definition of conditional probability

This rule tells us how to compute a joint probability using a conditional probability.

5.3 Definition of Independence (Special Case)

Two events a and b are **independent** if and only if:

$$P(b \mid a) = P(b)$$

That is, knowing that a happened does **not** change the probability of b.

Under this condition, the general rule simplifies to:

$$P(a \wedge b) = P(a) \cdot P(b)$$

⚠ **Important:** This simplified formula is valid **only when events are independent**.

5.4 Independent Events – Detailed Dice Example

Let:

- a = "first die is 6"
- b = "second die is 4"

Step 1: Individual probabilities

$$P(a) = 1/6 \quad P(b) = 1/6$$

Step 2: Conditional probability

$$P(b \mid a) = 1/6 \quad (\text{The second die is still uniformly random after seeing the first die.})$$

Step 3: Joint probability

$$P(a \wedge b) = P(a) \cdot P(b \mid a) = (1/6) \cdot (1/6) = 1/36$$

Conclusion:

- $P(b \mid a) = P(b)$
 - The events are **independent**
-

5.5 Dependent Events – Dice Example (Sum Constraint)

Let:

- $a = \text{"sum of the dice is 12"}$
- $b = \text{"first die is 6"}$

Step 1: Individual probability

$$P(b) = 1/6$$

Step 2: Conditional probability

If the sum is 12, the outcome **must** be (6,6).

So:

$$P(b \mid a) = 1$$

Step 3: Compare probabilities

$$P(b \mid a) \neq P(b)$$

Conclusion:

- Knowing a changes the probability of b
 - The events are **not independent**
-

5.6 Notes Very Important

✗ Incorrect reasoning:

$$P(a \wedge b) \neq P(a) \cdot P(b \mid a)$$

This is **never correct**.

The formula:

$$P(a \wedge b) = P(a) \cdot P(b \mid a)$$

is **always true**.

Independence is tested by checking:

$$P(b \mid a) = P(b)$$

—not by changing the multiplication rule.

5.7 Notes

“Conditional probability shrinks the world. Independence means shrinking the world does not change the chances.”

5.8 Python Example: Independent Case

```
# Independent events example
p_a = 1/6 # first die = 6
p_b = 1/6 # second die = 4
p_b_given_a = 1/6

p_joint = p_a * p_b_given_a
print("P(a ∧ b):", p_joint)
```

5.9 Python Example: Dependent Case

```
# Dependent events example
p_b = 1/6 # first die = 6
p_b_given_a = 1.0 # given sum = 12

print("Independent?", p_b_given_a == p_b)
```

5.10 Final Takeaway (Exam-Safe)

- $P(a \wedge b) = P(a) \cdot P(b \mid a) \rightarrow$ **always true**
 - Independence requires: $P(b \mid a) = P(b)$
 - Never assume independence without justification
 - Wrong independence assumptions lead to wrong AI models
-

6. Bayes' Rule

6.1 Why Bayes' Rule? (Intuition)

Bayes' Rule explains **how an intelligent system should change its belief when it receives new evidence**. In real life and in AI, we often observe an *effect* and want to reason about its *cause*.

In simple words:

Bayes' Rule answers the question: “Given what I just observed, what should I now believe?”

This is essential in AI systems such as medical diagnosis, fraud detection, spam filtering, and fault detection.

6.2 Bayes' Rule Formula

Bayes' Rule is defined as:

$$P(a | b) = (P(b | a) \cdot P(a)) / P(b)$$

Where:

- **P(a)** is the **prior**: belief about a *before* seeing evidence
 - **P(b | a)** is the **likelihood**: how likely the evidence is if a is true
 - **P(b)** is the **evidence**: how likely the evidence is overall
 - **P(a | b)** is the **posterior**: updated belief after seeing evidence
-

6.3 Why the Formula Makes Sense (Step-by-Step)

Bayes' Rule is not magic. It comes directly from conditional probability:

$$P(a | b) = P(a \wedge b) / P(b)$$

Using the multiplication rule:

$$P(a \wedge b) = P(b | a) \cdot P(a)$$

Substituting gives:

$$P(a | b) = (P(b | a) \cdot P(a)) / P(b)$$

This shows that Bayes' Rule is simply **reorganizing probabilities**, not inventing new ones.

6.4 Detailed Medical Test Example (Very Important)

Suppose:

- 1% of the population has a disease
- A test is 99% accurate

Define events:

- a = "person has the disease"
- b = "test result is positive"

Given: $P(a) = 0.01$ $P(\neg a) = 0.99$ $P(b | a) = 0.99$ $P(b | \neg a) = 0.01$

Step 1: Compute Evidence P(b)

$$P(b) = P(b | a)P(a) + P(b | \neg a)P(\neg a)$$

$$P(b) = (0.99 \cdot 0.01) + (0.01 \cdot 0.99) = 0.0198$$

Step 2: Apply Bayes' Rule

$$P(a | b) = (P(b | a) \cdot P(a)) / P(b)$$

$$P(a | b) = (0.99 \cdot 0.01) / 0.0198 \approx 0.5$$

Interpretation

Even though the test is **99% accurate**, a positive result means the person has the disease with **only 50% probability**.

6.5 Python Implementation: Medical Test

```
# Medical test example
P_a = 0.01          # disease
P_not_a = 0.99
P_b_given_a = 0.99
P_b_given_not_a = 0.01

# Evidence
P_b = P_b_given_a * P_a + P_b_given_not_a * P_not_a

# Posterior
P_a_given_b = (P_b_given_a * P_a) / P_b

print("P(disease | positive test):", P_a_given_b)
```

6.6 Dice Example (Connecting to Previous Sections)

Let:

- a = "sum = 12"
- b = "first die is 6"

Given: $P(a) = 1/36$ $P(b | a) = 1$ $P(b) = 1/6$

Bayes' Rule:

$$P(a | b) = (1 \cdot 1/36) / (1/6) = 1/6$$

This matches our earlier conditional probability result.

6.7 Why Bayes' Rule Is Used in AI

Bayes' Rule is used when:

- We observe symptoms and want causes

- We observe alerts and want explanations
- We update beliefs over time

It is the mathematical foundation of:

- Bayesian Networks
 - Naive Bayes classifiers
 - Probabilistic inference
-

6.8 Common Student Mistakes

1. Confusing $P(a | b)$ with $P(b | a)$
 2. Ignoring base rates (priors)
 3. Assuming high accuracy implies high certainty
-

6.9 Final Takeaway

Bayes' Rule provides a **principled way to update beliefs using evidence**. It ensures AI systems reason rationally instead of relying on intuition alone.

7. Joint Probability and Probability Rules

7.1 Why Do We Need Joint Probability? (Big Picture)

In real-world AI problems, we rarely care about a single variable in isolation. Instead, we want to reason about **multiple variables at the same time**.

Examples:

- Is it *cloudy* **and** *raining*?
- Is a network packet *malicious* **and** coming from an external IP?
- Is a patient *sick* **and** the test *positive*?

Joint probability allows us to model how variables **co-occur** in the same world.

Without joint probability:

- We cannot build Bayesian Networks
 - We cannot perform inference
 - We cannot connect causes and effects
-

7.2 Definition of Joint Probability

The **joint probability** of two events a and b , written as $P(a, b)$ or $P(a \wedge b)$, is the probability that **both events happen at the same time**.

Formally:

$P(a, b) = \sum P(\omega)$ for all worlds ω where both a and b are true

This definition directly comes from the idea of **possible worlds**.

7.3 Simple Dice Example (Step-by-Step)

Let:

- a = "first die is 6"
- b = "second die is 4"

There is exactly **one world** where both are true: (6,4).

Since all 36 worlds are equally likely:

$$P(a, b) = 1/36$$

This shows that joint probability is about **counting worlds where both conditions hold**.

7.4 Joint Probability Table (Very Important)

Joint probabilities are often represented using **tables**, especially when working with random variables.

Example: Weather model

Cloudy	Rain	Probability
Yes	Yes	0.08
Yes	No	0.32
No	Yes	0.02
No	No	0.58

Each row represents **one possible world**.

Check:

- All probabilities are ≥ 0
 - All probabilities sum to 1
-

7.5 Why Joint Tables Matter

Joint tables are the **foundation** of probabilistic reasoning:

- They contain *all information* about the system
- Any other probability can be derived from them

From a joint table, we can compute:

- Marginal probabilities

- Conditional probabilities
 - Independence relationships
-

7.6 Marginalization (Removing Variables)

Often, we are not interested in all variables. **Marginalization** allows us to "remove" variables by summing over them.

Example:

$$P(\text{Cloudy}) = P(\text{Cloudy}, \text{Rain}) + P(\text{Cloudy}, \text{No Rain})$$

Using the table:

$$P(\text{Cloudy}) = 0.08 + 0.32 = 0.40$$

7.7 Conditioning from Joint Probability

Conditional probabilities can be computed directly from joint probabilities.

Definition:

$$P(a \mid b) = P(a, b) / P(b)$$

Example:

$$\begin{aligned} P(\text{Rain} \mid \text{Cloudy}) &= P(\text{Cloudy}, \text{Rain}) / P(\text{Cloudy}) \\ &= 0.08 / 0.40 = 0.20 \end{aligned}$$

This shows how **joint probability connects directly to conditional probability**.

7.8 Why These Rules Are Needed in AI

Joint probability rules allow AI systems to:

- Combine multiple sources of uncertainty
- Answer complex queries (inference)
- Reason under incomplete information

They are essential for:

- Bayesian Networks
 - Hidden Markov Models
 - Probabilistic classifiers
-

7.9 Python Example: Joint Probability Table

```
# Joint probability table for weather
joint = {
    ("cloudy", "rain"): 0.08,
    ("cloudy", "no_rain"): 0.32,
    ("no_cloudy", "rain"): 0.02,
    ("no_cloudy", "no_rain"): 0.58
}

# Marginal probability of cloudy
p_cloudy = joint[("cloudy", "rain")] + joint[("cloudy", "no_rain")]
print("P(cloudy):", p_cloudy)

# Conditional probability of rain given cloudy
p_rain_given_cloudy = joint[("cloudy", "rain")] / p_cloudy
print("P(rain | cloudy):", p_rain_given_cloudy)
```

7.10 Common Student Mistakes

1. Thinking joint probability is multiplication by default
2. Forgetting to normalize probabilities
3. Mixing up $P(a, b)$ and $P(a | b)$

7.11 Final Takeaway

- Joint probability describes **what happens together**
- It is the most complete representation of uncertainty
- Marginalization and conditioning are derived from it
- Mastering joint probability is essential for understanding all later AI models

8. Bayesian Networks

8.1 What Is a Bayesian Network? (Big Picture)

A **Bayesian Network (BN)** is a graphical model that represents **how random variables depend on each other** and how uncertainty flows between them.

In simple words:

A Bayesian Network shows **what influences what**, and **by how much**, under uncertainty.

Bayesian Networks combine:

- **Probability theory** (to handle uncertainty)
- **Graphs** (to represent structure and causality)

They are one of the most important models in AI for reasoning, diagnosis, and decision making.

8.2 Why Do We Need Bayesian Networks?

Without Bayesian Networks, modeling real systems becomes impossible:

- Joint probability tables grow **exponentially** with variables
- Dependencies between variables are hard to track
- Inference becomes computationally infeasible

Bayesian Networks solve this by:

- Exploiting **conditional independence**
 - Breaking a big problem into **small local pieces**
 - Making reasoning **structured and interpretable**
-

8.3 Graph Structure (Nodes and Edges)

A Bayesian Network is a **directed acyclic graph (DAG)**:

- **Nodes** represent random variables
- **Directed edges** represent direct influence or dependency
- **No cycles** are allowed (a variable cannot depend on itself)

Graph intuition:

```
Rain → Traffic → Late
```

This means:

- Rain influences Traffic
 - Traffic influences being Late
-

8.4 Tree vs General Graph

Some Bayesian Networks form a **tree** (each node has at most one parent):

```
Rain
 |
Traffic
 |
Late
```

Others are **general directed graphs** (nodes can have multiple parents):

```
Rain → Traffic ← Accident
      |
```

▼
Late

Both are valid Bayesian Networks as long as the graph is acyclic.

8.5 Conditional Probability Tables (CPTs)

Each node X in a Bayesian Network has a **Conditional Probability Table (CPT)**:

$P(X \mid \text{Parents}(X))$

The CPT specifies how likely each value of X is, **given its parents**.

8.6 Example: Simple Weather Network (Step-by-Step)

Variables:

- Rain (Yes / No)
- Traffic (Heavy / Light)
- Late (Yes / No)

Graph:



Interpretation:

- Rain affects Traffic
- Traffic affects being Late

8.7 CPT Example (Explicit Numbers)

Prior probability:

$P(\text{Rain} = \text{Yes}) = 0.3$

CPT for Traffic:

Rain	Traffic=Heavy	Traffic=Light
Yes	0.7	0.3
No	0.2	0.8

CPT for Late:

Traffic	Late=Yes	Late=No
Heavy	0.9	0.1
Light	0.2	0.8

8.8 How the Joint Probability Is Computed

A Bayesian Network represents the joint probability as:

$$P(\text{Rain}, \text{Traffic}, \text{Late}) = P(\text{Rain}) \cdot P(\text{Traffic} \mid \text{Rain}) \cdot P(\text{Late} \mid \text{Traffic})$$

This factorization is what makes Bayesian Networks efficient.

8.9 Why This Factorization Is Powerful

Instead of storing a full joint table:

- $2 \times 2 \times 2 = 8$ entries

We store:

- 1 prior table
- 2 small CPTs

This idea scales to **dozens or hundreds of variables**.

8.10 Python Representation (Conceptual)

```
# Conceptual Bayesian Network structure
bn = {
    "Rain": [],
    "Traffic": ["Rain"],
    "Late": ["Traffic"]
}

# Conditional Probability Tables
P_Rain = {"Yes": 0.3, "No": 0.7}

P_Traffic_given_Rain = {
    "Yes": {"Heavy": 0.7, "Light": 0.3},
    "No": {"Heavy": 0.2, "Light": 0.8}
}

P_Late_given_Traffic = {
    "Heavy": {"Yes": 0.9, "No": 0.1},
    "Light": {"Yes": 0.2, "No": 0.8}
}

print(bn)
```

8.11 How Bayesian Networks Are Used in AI

Bayesian Networks are used for:

- Medical diagnosis (disease → symptoms)
 - Fault detection (cause → alarm)
 - Risk analysis
 - Natural language processing
 - Cybersecurity (attack → indicators)
-

8.12 Common Student Mistakes

1. Thinking arrows mean strict causality (they encode dependency)
 2. Forgetting that the graph must be acyclic
 3. Assuming independence without checking the graph
-

8.13 Final Takeaway

- Bayesian Networks model **complex uncertainty** using graphs
 - They exploit **conditional independence** to reduce complexity
 - Trees are a special case of Bayesian Networks
 - They are essential for scalable probabilistic inference in AI
-

9. Inference

9.1 What Is Inference? (Big Picture)

Inference is the process of answering questions about unknown variables **given what we already know**.

In probabilistic AI, inference answers queries of the form:

$$P(X \mid e)$$

Meaning:

What is the probability of X being true, given the observed evidence e ?

Inference is the **goal** of probabilistic models. Everything we built so far (probability, Bayes' Rule, Bayesian Networks) exists **to support inference**.

9.2 Why Do We Need Inference?

In real AI systems:

- We do NOT know everything
- Some variables are hidden

- We only observe partial evidence

Examples:

- Medical diagnosis: disease is hidden, symptoms are observed
- Cybersecurity: attack type is hidden, logs are observed
- Robotics: true state is hidden, sensors are noisy

Inference allows AI systems to:

- Reason under uncertainty
- Make decisions with incomplete information
- Update beliefs when new evidence arrives

9.3 Inference in Bayesian Networks (Graph View)

Inference works by **propagating probabilities through the graph**.

Example Bayesian Network:

Rain → Traffic → Late

Possible inference questions:

- $P(\text{Late} \mid \text{Rain} = \text{Yes})$
- $P(\text{Rain} \mid \text{Late} = \text{Yes})$
- $P(\text{Traffic} \mid \text{Late} = \text{Yes})$

The graph structure tells us **which variables influence others** and **how information flows**.

9.4 Query, Evidence, and Hidden Variables

Every inference problem has three parts:

- **Query variable (X):** what we want to know
- **Evidence (e):** what we have observed
- **Hidden variables (y):** variables not observed and not queried

Example:

Query: $P(\text{Rain} \mid \text{Late} = \text{Yes})$

- $X = \text{Rain}$
- $e = \text{Late} = \text{Yes}$
- $y = \text{Traffic}$

Hidden variables must be **summed out**.

9.5 Exact Inference by Enumeration (Core Idea)

The most basic inference algorithm is **enumeration**.

Formula:

$$P(X \mid e) = \alpha \sum P(X, e, y)$$

Where:

- y are hidden variables
- $\sum$ means summing over all their values
- α is a normalization constant

This formula directly comes from **joint probability**.

9.6 Enumeration Explained Step-by-Step

Steps:

1. Fix the query variable X
2. Fix the evidence e
3. Enumerate all possible values of hidden variables y
4. Compute joint probabilities for each case
5. Sum them
6. Normalize the result

This guarantees a **correct answer**, but may be slow.

9.7 Worked Example Using a Tree-Structured Network

Consider a tree Bayesian Network:

```
Rain
 |
Traffic
 |
Late
```

Query:

$$P(\text{Rain} \mid \text{Late} = \text{Yes})$$

Hidden variable:

- Traffic
-

Step 1: Write the joint probability

$$P(\text{Rain}, \text{Traffic}, \text{Late}) = P(\text{Rain}) \cdot P(\text{Traffic} \mid \text{Rain}) \cdot P(\text{Late} \mid \text{Traffic})$$

Step 2: Enumerate hidden variable Traffic

$$P(\text{Rain} \mid \text{Late}) \propto \sum P(\text{Rain}, \text{Traffic}, \text{Late})$$

Two cases:

- Traffic = Heavy
 - Traffic = Light
-

Step 3: Compute unnormalized probabilities

For Rain = Yes:

$$P(\text{Rain}=\text{Yes}, \text{Late}=\text{Yes}) = P(\text{Rain}=\text{Yes}) \cdot [P(\text{Traffic}=\text{Heavy} \mid \text{Rain}=\text{Yes}) \cdot P(\text{Late}=\text{Yes} \mid \text{Heavy})$$

- $P(\text{Traffic}=\text{Light} \mid \text{Rain}=\text{Yes}) \cdot P(\text{Late}=\text{Yes} \mid \text{Light})]$

Repeat for Rain = No.

Step 4: Normalize

Divide by the sum of both cases so probabilities add to 1.

This produces the final posterior distribution.

9.8 Graphical Intuition (Very Important)

You can explain inference visually:

Evidence flows backward and forward through the graph
Hidden nodes block or transmit information

- Evidence at a leaf node (Late) affects ancestors (Traffic, Rain)
 - This is why Bayesian Networks support **diagnostic reasoning**
-

9.9 Python Example: Enumeration (Conceptual)

```
# Conceptual enumeration example
# Suppose unnormalized probabilities
p_yes = 0.12
p_no = 0.08

# Normalize
alpha = 1 / (p_yes + p_no)
```

```
print("P(Rain=Yes | Late=Yes):", p_yes * alpha)
print("P(Rain=No | Late=Yes):", p_no * alpha)
```

9.10 Why Exact Inference Is Hard

Exact inference becomes difficult when:

- Many hidden variables exist
- Networks are large
- Graphs are densely connected

Number of terms grows **exponentially**.

This motivates **approximate inference methods**.

9.11 Tree vs General Graph Inference

- **Tree-structured networks:** inference is efficient
- **General graphs:** inference is expensive

This difference explains why:

- Naive Bayes is fast
 - Loopy Bayesian Networks are hard
-

9.12 Common Student Mistakes

1. Forgetting to sum over hidden variables
 2. Forgetting to normalize
 3. Mixing joint and conditional probabilities
 4. Ignoring the graph structure
-

9.13 Final Takeaway

- Inference answers probabilistic questions
 - It uses joint probability + marginalization
 - Enumeration is exact but slow
 - Graph structure guides computation
 - Inference is the core task of probabilistic AI
-

10. Approximate Inference

10.1 Why Do We Need Approximate Inference?

Exact inference (such as enumeration) is **guaranteed to be correct**, but it becomes **computationally infeasible** when:

- The Bayesian Network has many variables
- There are many hidden variables
- The graph is densely connected

In such cases, exact inference may require **exponential time**.

Approximate inference provides a **practical solution**:

Instead of computing exact probabilities, we estimate them using random samples.

This trade-off is fundamental in AI:

- **Exact inference** → correct but slow
 - **Approximate inference** → fast but approximate
-

10.2 Core Idea of Sampling (Intuition)

Sampling works by:

1. Generating many possible worlds (samples)
2. Counting how often events occur
3. Using frequencies as probability estimates

Intuition:

Probability is approximated by how often something happens in many trials.

As the number of samples increases, the approximation becomes more accurate.

10.3 Sampling as a Tree / Graph Process

In a Bayesian Network, sampling follows the **graph structure**.

For example:

Rain → Traffic → Late

Sampling proceeds top-down:

1. Sample Rain from $P(\text{Rain})$
2. Sample Traffic from $P(\text{Traffic} \mid \text{Rain})$
3. Sample Late from $P(\text{Late} \mid \text{Traffic})$

Each full assignment corresponds to **one possible world**.

10.4 Rejection Sampling (Step-by-Step)

Rejection sampling generates samples from the full joint distribution, then **rejects samples that do not match the evidence**.

Steps:

1. Generate a complete sample from the network
 2. Check whether the sample agrees with evidence
 3. Keep the sample if it matches; otherwise discard it
 4. Estimate probabilities using the remaining samples
-

10.5 Rejection Sampling Example (Dice Analogy)

Suppose we want:

$P(\text{sum} = 12 \mid \text{first die} = 6)$

Procedure:

- Generate many dice rolls
- Reject rolls where first die $\neq 6$
- Count how often sum = 12 in remaining rolls

This directly mirrors conditional probability.

10.6 Rejection Sampling – Python Example

```
import random

def rejection_sampling(num_samples=10000):
    accepted = 0
    success = 0
    for _ in range(num_samples):
        d1 = random.randint(1, 6)
        d2 = random.randint(1, 6)
        if d1 == 6:           # evidence
            accepted += 1
            if d1 + d2 == 12: # query
                success += 1
    return success / accepted if accepted > 0 else 0

print("Estimated P(sum=12 | first die=6):", rejection_sampling())
```

10.7 Problem with Rejection Sampling

Rejection sampling becomes inefficient when:

- Evidence is rare
- Most samples are rejected

Example:

- $P(\text{first die} = 6) = 1/6 \rightarrow 5/6$ samples wasted
- In large networks, waste becomes extreme

This motivates **Likelihood Weighting**.

10.8 Likelihood Weighting (Key Idea)

Likelihood weighting avoids rejection by:

- **Fixing evidence variables**
- Sampling only non-evidence variables
- Assigning a **weight** to each sample

Weight = probability of evidence given the sampled parents

10.9 Likelihood Weighting – Graph View

Given evidence:

Late = Yes

Sampling:

1. Fix Late = Yes
2. Sample Traffic
3. Sample Rain
4. Weight sample by $P(\text{Late} = \text{Yes} \mid \text{Traffic})$

Samples with higher likelihood contribute more.

10.10 Likelihood Weighting – Python Example

```
import random

def likelihood_weighting(num_samples=10000):
    weighted_sum = 0
    weight_total = 0
    for _ in range(num_samples):
        # sample Rain
        rain = "Yes" if random.random() < 0.3 else "No"

        # sample Traffic given Rain
        if rain == "Yes":
            traffic = "Heavy" if random.random() < 0.7 else "Light"
        else:
            traffic = "Heavy" if random.random() < 0.2 else "Light"
```

```
# evidence: Late = Yes
weight = 0.9 if traffic == "Heavy" else 0.2

if rain == "Yes":
    weighted_sum += weight
    weight_total += weight

return weighted_sum / weight_total

print("Estimated P(Rain=Yes | Late=Yes):", likelihood_weighting())
```

10.11 Comparison: Exact vs Approximate Inference

Method	Accuracy	Speed	When to Use
Enumeration	Exact	Slow	Small networks
Rejection Sampling	Approx	Fast	Common evidence
Likelihood Weighting	Approx	Faster	Rare evidence

10.12 Common Student Mistakes

1. Forgetting to normalize weighted samples
2. Using rejection sampling with rare evidence
3. Thinking approximate inference is inaccurate by default

10.13 Final Takeaway

- Approximate inference trades accuracy for speed
- Sampling simulates possible worlds
- Graph structure guides sampling order
- Likelihood weighting fixes rejection inefficiency
- Approximate inference enables probabilistic AI at scale

11. Uncertainty Over Time

11.1 Why Uncertainty Over Time Matters (Big Picture)

Many real-world AI problems are **not static**. The world changes over time, and an AI system must continuously update its beliefs as time passes.

Examples:

- Weather today affects weather tomorrow
- A robot’s current position affects its future position
- A system state evolves as actions are taken

Uncertainty over time means:

- The current state is not perfectly known
- The future state is uncertain
- Observations are noisy or incomplete

To reason in such environments, we need **temporal probabilistic models**.

11.2 Markov Assumption (Core Idea)

The **Markov Assumption** simplifies reasoning over time by stating:

The future depends only on the present, not on the entire past.

Formally:

$$P(X_t | X_{t-1}, X_{t-2}, \dots, X_0) = P(X_t | X_{t-1})$$

This assumption allows us to model long sequences efficiently.

11.3 Why We Need the Markov Assumption

Without the Markov assumption:

- We would need to remember the entire history
- Computation would grow exponentially
- Learning and inference would be infeasible

With the Markov assumption:

- State representation is compact
 - Inference is tractable
 - Models scale to long time horizons
-

11.4 Markov Chains (Detailed Explanation)

A **Markov Chain** is a sequence of random variables:

$X_0, X_1, X_2, \dots$

that satisfies the Markov assumption.

Each state depends only on the previous state.

11.5 Markov Chain as a Graph

Markov Chains can be represented as a **temporal graph**:

$X_0 \rightarrow X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow \dots$

Each arrow represents a **transition probability** between states.

11.6 Example: Weather Markov Chain (Step-by-Step)

States:

- Sun
- Rain

Transition model:

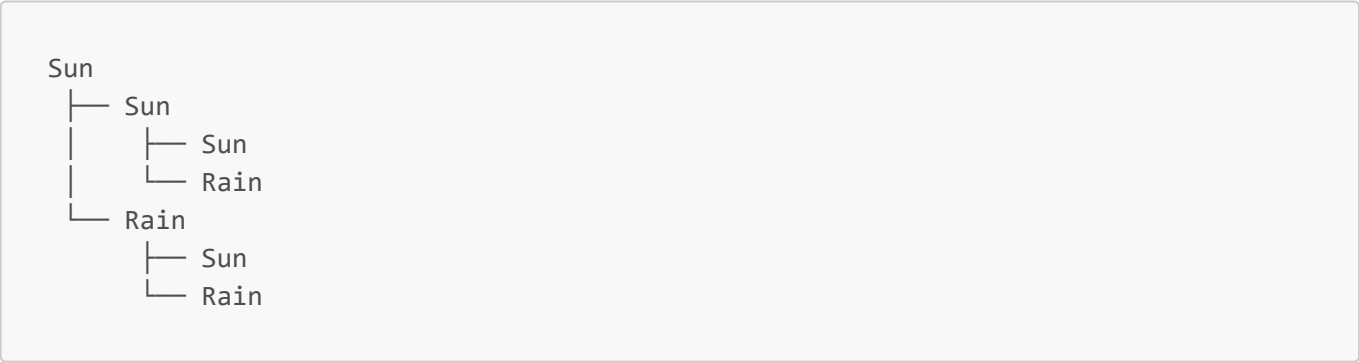
From \ To	Sun	Rain
Sun	0.8	0.2
Rain	0.3	0.7

Interpretation:

- If today is sunny, tomorrow is likely sunny
 - If today is rainy, rain may persist
-

11.7 Tree Interpretation Over Time

Over time, the Markov Chain unfolds into a **tree of possibilities**:



Each path represents one possible evolution of the system.

11.8 Python Example: Markov Chain Transition

```
transition = {
    "sun": {"sun": 0.8, "rain": 0.2},
    "rain": {"sun": 0.3, "rain": 0.7}
}

current_state = "sun"
print("Transition probabilities from sun:", transition[current_state])
```

11.9 Hidden Markov Models (HMMs) – Why We Need Them

In many problems, the true state is **hidden**.

Examples:

- Weather is hidden, sensors give noisy readings
- Speech phonemes are hidden, audio is observed
- User intention is hidden, clicks are observed

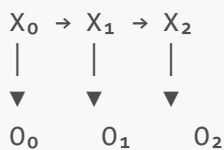
Hidden Markov Models extend Markov Chains by adding **observations**.

11.10 Structure of a Hidden Markov Model

An HMM consists of:

1. **Hidden states** X_t (Markov Chain)
2. **Observations** O_t (what we see)
3. **Transition model** $P(X_t | X_{t-1})$
4. **Emission model** $P(O_t | X_t)$

Graphically:



11.11 Example: Weather with Noisy Sensor

Hidden states:

- Sun
- Rain

Observations:

- Dry
- Wet

Emission model:

State	Dry	Wet
Sun	0.9	0.1
Rain	0.2	0.8

This means sensors are not perfect.

11.12 Why HMMs Are Needed

Without HMMs:

- We cannot reason about hidden processes
- Observations would be treated as truth

With HMMs:

- We separate **state** from **observation**
 - We reason probabilistically about uncertainty
-

11.13 Python Example: HMM Models

```
# Transition model
transition = {
    "sun": {"sun": 0.8, "rain": 0.2},
    "rain": {"sun": 0.3, "rain": 0.7}
}

# Emission model
emission = {
    "sun": {"dry": 0.9, "wet": 0.1},
    "rain": {"dry": 0.2, "wet": 0.8}
}

print("Transition:", transition)
print("Emission:", emission)
```

11.14 Connection to AI Tasks

Uncertainty over time is central to:

- Robotics (localization, tracking)
 - Speech recognition
 - Time-series prediction
 - Cybersecurity (attack progression)
-

11.15 Final Takeaway

- Time adds a new dimension of uncertainty
- Markov assumption makes temporal reasoning feasible
- Markov Chains model visible states
- HMMs model hidden states with noisy observations

- These models are essential for dynamic AI systems
-

12. Tasks in HMMs

- Filtering: current state
 - Prediction: future state
 - Smoothing: past state
 - Most likely explanation
-

13. Summary

Key Takeaways

- Probability formalizes uncertainty
 - Conditional probability updates beliefs
 - Bayes' Rule reverses reasoning
 - Bayesian Networks encode structure
 - Inference answers questions under uncertainty
 - Markov models handle time
-

14. Programming Assignment: Uncertainty in AI (Python)

Assignment Title

Implementing Probabilistic Reasoning under Uncertainty

14.1 Learning Objectives

After completing this assignment, students should be able to:

- Translate uncertainty problems into probabilistic models
 - Work with possible worlds and probability distributions
 - Implement conditional probability and Bayes' Rule in Python
 - Model uncertainty using a Bayesian Network
 - Perform simple inference by enumeration
 - Understand *why* probabilistic reasoning is needed in AI systems
-

14.2 Problem Description (Context)

You are asked to build a **small AI reasoning system** that deals with uncertainty using probability.

Scenario:

A person may arrive **late** to work depending on whether it is **raining** and how **traffic** behaves. However, you cannot observe all variables directly.

14.3 Variables and Their Meaning

Define the following random variables:

- **Rain** $\in \{\text{Yes, No}\}$
- **Traffic** $\in \{\text{Heavy, Light}\}$
- **Late** $\in \{\text{Yes, No}\}$

Interpretation:

- Rain influences Traffic
- Traffic influences whether the person is Late

Graph structure (Bayesian Network):

Rain $\rightarrow$ Traffic $\rightarrow$ Late

14.4 Given Probability Tables (DO NOT CHANGE)

Prior probability:

$$P(\text{Rain} = \text{Yes}) = 0.3 \quad P(\text{Rain} = \text{No}) = 0.7$$

Conditional Probability: Traffic given Rain

Rain	Traffic=Heavy	Traffic=Light
Yes	0.7	0.3
No	0.2	0.8

Conditional Probability: Late given Traffic

Traffic	Late=Yes	Late=No
Heavy	0.9	0.1
Light	0.2	0.8

14.5 Tasks to Implement (Step-by-Step)

Task 1: Define the Probability Model (20%)

- Represent the above probabilities in Python using dictionaries
- Clearly comment each probability table

Expected outcome:

- Python data structures correctly representing the model
-

Task 2: Compute Joint Probability (20%)

Write a function that computes:

$P(\text{Rain}, \text{Traffic}, \text{Late})$

using the formula:

$P(\text{Rain}, \text{Traffic}, \text{Late}) = P(\text{Rain}) \cdot P(\text{Traffic} \mid \text{Rain}) \cdot P(\text{Late} \mid \text{Traffic})$

Test your function on at least **two different combinations**.

Task 3: Conditional Probability (Inference) (25%)

Compute the probability:

$P(\text{Rain} = \text{Yes} \mid \text{Late} = \text{Yes})$

Steps students must follow:

1. Identify hidden variables
2. Enumerate all their values
3. Compute unnormalized probabilities
4. Normalize the result

Students must **explain each step in comments**.

Task 4: Why Conditional Probability Is Needed (15%)

In comments or a short text block, explain:

- Why $P(\text{Rain} \mid \text{Late})$ is different from $P(\text{Rain})$
- How observing evidence changes the probability space

This part is conceptual and graded on explanation quality.

Task 5: Optional Extension (Bonus 10%)

Implement **rejection sampling** to estimate:

$P(\text{Rain} = \text{Yes} \mid \text{Late} = \text{Yes})$

Compare your estimated value with the exact result.

14.6 Starter Code (Provided)

```
# PRIOR
P_Rain = {"Yes": 0.3, "No": 0.7}

# CPT: Traffic | Rain
P_Traffic_given_Rain = {
    "Yes": {"Heavy": 0.7, "Light": 0.3},
    "No": {"Heavy": 0.2, "Light": 0.8}
}

# CPT: Late | Traffic
P_Late_given_Traffic = {
    "Heavy": {"Yes": 0.9, "No": 0.1},
    "Light": {"Yes": 0.2, "No": 0.8}
}
```

14.7 Submission Requirements

- One Python file ([uncertainty.py](#))
- Clear comments explaining logic
- Output of computed probabilities